

EcoRouter — Teleprompter Script v2

Ashwin · CS 153 · Automation / Agent Systems · One-Person Frontier Lab · June 2026

[0:00 – 0:25] OPENING

Hi, I'm Ashwin.

This is **EcoRouter Agent** — an autonomous system that routes AI compute workloads to the greenest cloud regions in real time, while respecting urgency deadlines, SLA windows, and data residency rules.

I built it for Stanford CS 153, **Automation and Agent Systems** track, as my **One-Person Frontier Lab** project.

[0:25 – 1:05] VIDEO Q1 + PROBLEM & INSIGHT Q1 (3 PTS)

Why did I build this?

As AI scales, the bottleneck is not only GPUs — it's **electricity**. Training and batch inference consume massive power, and grid carbon intensity — grams of CO₂ per kilowatt-hour — **changes by region and by hour** as solar, wind, and demand shift.

Today, a DevOps or FinOps team manually watches grid data and decides where jobs run. That does not scale.

EcoRouter's insight: replace that manual scheduler with an **intelligent agent** — a digital twin infrastructure manager that reads

telemetry, evaluates constraints, and dispatches each job to the best region it can legally reach.

[1:05 – 2:10] VIDEO Q2 + EXECUTION & TECHNICAL WORK Q2 (5 PTS)

How exactly does the product work? For an agent-systems project, here is the full stack.

Layer one — Grid simulator: thirty real AWS regions with simulated carbon and dollar-per-kilowatt-hour tariffs. Optional Electricity Maps integration for live data.

Layer two — Job queue: realistic enterprise workloads — fine-tuning, RAG batches, GDPR-locked EU inference, HIPAA US jobs. Each job has compute hours, urgent versus flexible priority, and optional locality locks.

Layer three — Agent brain in `ecorouter dot py`: Gemini two-point-five Flash with **LLM tool-calling** — get grid carbon intensity, and assign workload. The agent reasons over telemetry, then executes assignments. Plus production routers: **Pareto**, cost-aware, load-balanced, and **forecast-aware deferral** over a twelve-hour window. A mock heuristic fallback guarantees the demo never breaks.

Layer four — Interfaces: this Streamlit dashboard, a **FastAPI REST layer**, CLI agent loop, and ESG PDF export.

Same logic runs from CLI, API, or UI — fully reproducible from the public GitHub repo with `requirements dot txt`.

[2:10 – 2:35] DEMO — SETUP

Let me show it live.

At the top: personalized workspace stats — thirty regions, today's greenest region, and carbon upside versus a static us-east-1 baseline.

Scroll the **live carbon chart** — low-carbon Nordic and Canadian hydro versus high-carbon APAC coal regions.

The **job queue** shows realistic scenarios: most jobs are carbon-flexible; a few are locked for GDPR or US latency compliance.

Sidebar: I'll run **Pareto routing** with **forecast-aware scheduling** enabled.

[2:35 – 3:20] DEMO — OPTIMIZATION

Done. Look at the **Dispatch Summary**.

EcoRouter Carbon and **Carbon Saved** are in **grams of CO₂**, with concrete totals.

Jobs routed across **multiple green regions** — not one datacenter hotspot — because Pareto load-spreading balances carbon and cost.

The **assignment table** shows baseline versus EcoRouter per job: green rows mean region changed with carbon savings; amber means rerouted; gray means unchanged.

→ *CLICK: Run EcoRouter Optimization · Point at gCO₂ stat cards · Scroll assignment table*

[3:20 – 3:50] DEMO — A/B & FORECAST

Switch to **A/B Comparison**.

EcoRouter is measured against a naive baseline — static us-east-1 or round-robin. You see total carbon, cost in dollars, and **percent savings**. Often fifty to ninety percent carbon reduction on flexible batches.

Open **Forecast and Deferral**: twelve-hour per-region forecast and deferral recommendations for flexible jobs.

→ *CLICK: A/B Comparison tab · Forecast tab*

[3:50 – 4:15] VIDEO Q3 — USE CASES & IMPACT Q3

Who uses this and why does it matter?

Cloud providers and AI labs plug it into batch schedulers to cut Scope 2 emissions without breaking SLAs.

FinOps teams trade carbon against cost with Pareto and cost-aware modes.

DePIN and distributed compute networks apply the same logic wherever workloads are geographically mobile.

The impact: greener AI infrastructure without asking humans to manually chase grid data every hour.

[4:15 – 4:45] EVALUATION & EVIDENCE (3 PTS)

How do we know it works?

First, **constraint satisfaction**: locality-locked jobs never leave their required region. Urgent jobs respect SLA deadlines.

Second, **quantified savings**: the A/B tab reports grams of CO₂ and dollars saved versus baseline, with per-job breakdowns.

Third, **iteration and limitations**: SDK migration to google-genai, four to thirty regions, public commit history — documented in the README. This is a **simulation**, not production Kubernetes yet.

The **Tools and API** tab shows curl examples — graders can hit slash api slash v1 slash optimize without the UI.

[4:45 – 5:10] COMMUNICATION & PRESENTATION (2 PTS)

For anyone outside the project: the README documents architecture, quick start, and this rubric mapping. The **TA Access Guide** gives a three-minute grading path. Clone, install, and run with mock mode — no API key required. This demo walks tab by tab so the system is understandable and reproducible.

[5:10 – 5:30] VIDEO Q4 — FUTURE WORK Q4

What more would I add?

A production **Kubernetes operator**, reinforcement learning over multi-day carbon forecasts, and per-tenant carbon budgets for enterprise ESG reporting.

[5:30 – 5:50] PROCESS, INTEGRITY & DISCLOSURE (2 PTS)

I used **Cursor and Gemini** as accelerators — fully disclosed in the README per CS 153 AI policy. All simulation logic and routing

design are original; no forked base repo. Limitations — simulated grid, mock fallback, no live AWS dispatch — are documented openly.

[5:50 – 6:00] CLOSE

The full project is open source:

github.com/ashwin122705/ecorouter-agent

TAs can run it locally with streamlit run app dot py, or use the deployed demo link in the README.

Thank you for watching.

→ Show GitHub URL + README rubric section. End recording.

Action Cues — Glance Only

Time	On-Screen Action
0:20	Hero stats: regions, greenest, gCO ₂
2:15	Scroll carbon bar chart
2:25	Job queue — Locality column
2:30	Sidebar: Pareto + Forecast ON
2:35	CLICK: Run EcoRouter Optimization
2:50	Dispatch Summary gCO ₂ cards
3:22	A/B Comparison tab
3:40	Forecast tab
4:20	Tools & API tab — rubric expander
5:52	GitHub + README rubric section

Rubric + Video Checklist

Requirement	Covered at
Problem & Insight (3 pts)	0:25 · Video Q1
Execution & Technical Work (5 pts)	1:05 · Video Q2 + demo
Evaluation & Evidence (3 pts)	4:15 · A/B tab
Communication & Presentation (2 pts)	4:45 · README / TA guide
Process & Disclosure (2 pts)	5:30 · AI policy
Video Q3 — use cases	3:50
Video Q4 — future work	5:10

